

System Documentation — Student-Portal-DEPI

1. Overview

Student-Portal-DEPI is a role-based student management portal with three user roles: **student**, **instructor**, and **admin**. Students can enroll in courses, view schedules, submit assignments, and check grades. Instructors can manage their assigned courses, create assignments, post announcements, and maintain gradebooks. Admins have full control over user management, course creation (including instructor assignment and scheduling), and system-wide announcements.

Tech Stack

Dependency	Version
React	^19.2.7
React DOM	^19.2.7
React Router DOM	^7.18.1
Vite	^8.1.1
Tailwind CSS	^4.3.2
@tailwindcss/vite	^4.3.2
@supabase/supabase-js	^2.110.5
Zod	^4.4.3
lucide-react	^1.24.0
@vitejs/plugin-react	^6.0.3

The project uses plain JSX (no TypeScript), ESM modules (`"type": "module"`), and Tailwind CSS v4 with `@import "tailwindcss"` syntax.

2. Architecture

Folder Structure

src/	
main.jsx	– Entry point; renders <code><App /></code> inside <code>StrictMode</code>
App.jsx	– Router definition + <code>AuthProvider</code> wrapper
index.css	– Tailwind CSS v4 entry (<code>@import "tailwindcss"</code>)
context/	– <code>AuthContext</code> (authentication state provider)
components/	– Shared UI: <code>AppLayout</code> , <code>SideNavBar</code> , <code>TopAppBar</code> , <code>ProtectedRoute</code> , <code>StatCard</code> , <code>Badge</code> , <code>CourseCard</code> , <code>DataTable</code>
pages/	– Route-level page components (14 pages)
api/	– Supabase API layer (8 files, one per resource domain)
validation/	– Zod schemas for form validation (6 files)
lib/	– <code>supabaseClient.js</code> (shared Supabase client instance)

Request Flow

1. **Routing:** `App.jsx` wraps everything in `<BrowserRouter>` → `<AuthProvider>` → `<Routes>` .
 2. **Public route:** `/login` renders `LoginPage` with no layout or protection.
 3. **Protected routes:** All other routes are nested inside `<AppLayout>` which provides the sidebar + top bar shell.
 4. **Role gating:** `<ProtectedRoute role="student|instructor|admin">` checks authentication and role before rendering child routes via `<Outlet />` .
 5. **Page components:** Each page imports functions from `src/api/*.js` to fetch/mutate data.
 6. **API layer:** Each `api/*.js` file exports plain `async` function calls that use the shared Supabase client from `src/lib/supabaseClient.js` . Components never talk to Supabase directly.
 7. **Supabase:** The backend is a hosted Supabase Postgres database accessed via its auto-generated REST API (PostgREST).
-

3. Authentication & Authorization

AuthContext (`src/context/AuthContext.jsx`)

- **Session persistence:** On app load, a `useEffect` reads `localStorage` key `spd_user` . If a saved user object exists, it is parsed and set as the current user.
- **login(email, password)** : Queries the Supabase `users` table for a row matching the given email (via `.maybeSingle()`). If no row is found, throws `"No account found with that email."` . If the row's `password` field does not match the provided password, throws `"Invalid email or password."` . On success, constructs a user object `{ id, email, role, name, phone, bio }` , stores it in `localStorage` , updates the user's `last_login` field, and returns the user object.
- **logout()** : Sets user to `null` and removes the `spd_user` key from `localStorage` .
- **isLoggedIn** : Derived boolean (`user !== null`).
- **loading** : Boolean that is `true` during the initial `localStorage` check, then `false` .

ProtectedRoute (`src/components/ProtectedRoute.jsx`)

Performs three checks in order:

1. **Loading:** If `loading` is `true` , renders `<p>Loading...</p>` .
2. **Not logged in:** If `!isLoggedIn` , redirects to `/login` .
3. **Wrong role:** If a `role` prop is provided and `user.role !== role` , redirects to `/ ${user.role} /dashboard` (sends the user to their own role's dashboard).
4. If all checks pass, renders `<Outlet />` (the child route).

Known Limitation

Authentication queries the `users` table directly and compares passwords in **plaintext**. There is no password hashing (bcrypt, argon2, etc.). This is acceptable for a development/demo stage but must be replaced with proper hashing before any production deployment. Supabase Auth (GoTrue) is not used.

4. Routing Map

Route Path	Page Component	Required Role
<code>/login</code>	<code>LoginPage</code>	Public (no auth)
<code>/student/dashboard</code>	<code>StudentDashboard</code>	student

/student/courses	MyCourses	student
/student/courses/:courseId	CourseDetail	student
/student/notifications	Notifications	student
/student/schedule	Schedule	student
/student/grades	Grades	student
/instructor/dashboard	InstructorDashboard	instructor
/instructor/courses/:courseId	CourseManagement	instructor
/instructor/courses/:courseId/gradebook	Gradebook	instructor
/admin/dashboard	AdminDashboard	admin
/admin/users	UserManagement	admin
/admin/courses/new	AdminCourseCreate	admin
/profile	Profile	Any logged-in user
/notifications	Notifications	Any logged-in user
* (catch-all)	Redirects to /login	—

All authenticated routes are wrapped in `<AppLayout>` which provides the `SideNavBar` and `TopAppBar`.

5. Database Schema

All tables live in a single Supabase (Postgres) project. RLS (Row Level Security) is disabled on all tables — the anon key has full read/write access. This is intentional for the current development stage.

users

Stores all portal users (students, instructors, admins). Also used for authentication.

Column	Type	Notes
id	bigint	PK, auto-generated identity
name	text	NOT NULL
email	text	NOT NULL, UNIQUE
role	text	NOT NULL, CHECK IN ('student', 'instructor', 'admin')
status	text	NOT NULL, DEFAULT 'Active', CHECK IN ('Active', 'Inactive', 'Offline', 'Pending')
last_login	text	
avatar_url	text	
initials	text	

phone	text	
bio	text	
password	text	Added via migration-v2.sql; stored in plaintext

Pages: LoginPage (read), UserManagement (full CRUD), Profile (edit), AuthContext (read for login).

notifications

User notifications (academic alerts, system announcements, registration confirmations).

Column	Type	Notes
id	bigint	PK, auto-generated identity
user_id	bigint	FK → users(id). NULL for system-wide announcements
day	text	NOT NULL (e.g., 'Today', 'Yesterday')
category_id	text	NOT NULL (e.g., 'academic', 'system', 'events')
type	text	NOT NULL, CHECK IN ('urgent', 'success', 'system', 'event')
source_label	text	NOT NULL
title	text	NOT NULL
description	text	NOT NULL
time_label	text	NOT NULL
action_label	text	
read	boolean	NOT NULL, DEFAULT false

Pages: Notifications (read + mark-read), TopAppBar (read last 3), AdminDashboard (create system announcements), InstructorDashboard (create course announcements), MyCourses (create on registration).

schedule_classes

Weekly timetable entries for courses.

Column	Type	Notes
id	bigint	PK, auto-generated identity
day	text	NOT NULL (e.g., 'Mon', 'Tue')
course_code	text	NOT NULL
start_hour	numeric	NOT NULL (24h decimal, e.g., 9.5 = 9:30 AM)
end_hour	numeric	NOT NULL
time_label	text	NOT NULL

course_id	text	FK → courses(id), added via migration-v2.sql
-----------	------	--

Pages: `Schedule` (read), `StudentDashboard` (read, filtered by enrolled courses), `AdminCourseCreate` (create).

semesters

Column	Type	Notes
id	text	PK (e.g., 'fall-2024')
label	text	NOT NULL

Pages: `Grades` (read).

semester_courses

Individual course grades per semester (transcript data).

Column	Type	Notes
id	bigint	PK, auto-generated identity
semester_id	text	FK → semesters(id) ON DELETE CASCADE
name	text	NOT NULL
code	text	NOT NULL
subject	text	NOT NULL
credits	integer	NOT NULL
grade	text	NOT NULL

Pages: `Grades` (read), `StudentDashboard` (read recent grades).

courses

Master course catalog.

Column	Type	Notes
id	text	PK (e.g., 'cs301')
title	text	NOT NULL
meta	text	
description	text	
department	text	
term	text	
credits	integer	

instructor_name	text	
instructor_role	text	
instructor_email	text	
instructor_office	text	
instructor_hours	text	
instructor_id	bigint	FK → users(id), added via migration-v2.sql

Pages: CourseManagement (read), CourseDetail (read), MyCourses (read catalog), InstructorDashboard (read, filtered by instructor_id), AdminCourseCreate (create), AdminDashboard (read count).

course_students

Enrollment roster per course.

Column	Type	Notes
id	text	PK (e.g., '1029384' or 'reg-{userId}-{courseId}')
course_id	text	FK → courses(id) ON DELETE CASCADE
name	text	NOT NULL
initials	text	
status	text	DEFAULT 'Active', CHECK IN ('Active', 'At Risk')
user_id	bigint	FK → users(id), added via migration-v2.sql

Pages: CourseManagement (read), Gradebook (read), InstructorDashboard (read for student counts + announcement targeting), MyCourses (create on registration).

course_sessions

Upcoming class sessions per course.

Column	Type	Notes
id	bigint	PK, auto-generated identity
course_id	text	FK → courses(id) ON DELETE CASCADE
month	text	NOT NULL
day	text	NOT NULL
title	text	NOT NULL
meta	text	
tone	text	DEFAULT 'default'

Pages: CourseManagement (read).

course_weeks

Weekly structure within a course (for CourseDetail materials view).

Column	Type	Notes
id	bigint	PK, auto-generated identity
course_id	text	FK → courses(id) ON DELETE CASCADE
title	text	NOT NULL
date_range	text	

Pages: CourseDetail (read).

course_materials

Files/resources within a course week.

Column	Type	Notes
id	bigint	PK, auto-generated identity
week_id	bigint	FK → course_weeks(id) ON DELETE CASCADE
type	text	NOT NULL, CHECK IN ('pdf', 'video', 'code')
name	text	NOT NULL
info	text	
action	text	

Pages: CourseDetail (read).

course_files

Recent files sidebar in CourseDetail.

Column	Type	Notes
id	bigint	PK, auto-generated identity
course_id	text	FK → courses(id) ON DELETE CASCADE
name	text	NOT NULL
type	text	NOT NULL

Pages: CourseDetail (read).

gradebook_entries

Per-student grade records within a course.

Column	Type	Notes
id	text	PK (e.g., '98210')
course_id	text	FK → courses(id) ON DELETE CASCADE
name	text	NOT NULL
initials	text	
overall	text	
hw1	numeric	
quiz1	numeric	
midterm	numeric	
hw2	numeric	
attention	boolean	DEFAULT false
excused	boolean	DEFAULT false

Pages: `Gradebook` (read + update), `InstructorDashboard` (read for avg grade computation).

assignments

Instructor-created assignments per course.

Column	Type	Notes
id	bigint	PK, auto-generated identity
course_id	text	FK → courses(id) ON DELETE CASCADE
title	text	NOT NULL
description	text	
due_date	text	
created_at	timestamp	DEFAULT now()

Pages: `CourseManagement` (create + delete), `CourseDetail` (read), `InstructorDashboard` (read for pending grading count), `StudentDashboard` (read for action-required count).

assignment_submissions

Student submissions for assignments.

Column	Type	Notes
id	bigint	PK, auto-generated identity
assignment_id	bigint	FK → assignments(id) ON DELETE CASCADE

user_id	bigint	FK → users(id)
submission_text	text	
file_url	text	Added via migration-v2.sql; URL to external file
submitted_at	timestamp	DEFAULT now()

Pages: CourseDetail (create submission, read own submissions).

instructor_active_courses

Legacy table for instructor dashboard course cards. Currently the InstructorDashboard fetches from `courses` filtered by `instructor_id` instead.

Column	Type	Notes
id	text	PK
code	text	
name	text	NOT NULL
meta	text	
students	integer	
avg_grade	text	
color	text	

Pages: InstructorDashboard (via `getInstructorActiveCourses` in `dashboardApi.js`, but currently unused — dashboard uses `getAllCourses` + filter).

instructor_alerts

Alert items shown on the instructor dashboard sidebar.

Column	Type	Notes
id	bigint	PK, auto-generated identity
title	text	NOT NULL
meta	text	
dot_color	text	

Pages: InstructorDashboard (read).

admin_stats

Legacy table for admin dashboard stat cards. Currently the AdminDashboard computes stats from `users` and `courses` tables instead.

Column	Type	Notes
id	bigint	PK, auto-generated identity
label	text	NOT NULL
value	text	NOT NULL
delta	text	
delta_type	text	CHECK IN ('up', 'down')

Pages: AdminDashboard (via `getAdminStats` in `dashboardApi.js`, but currently unused — dashboard computes from `users/courses`).

admins

Legacy table for the admin dashboard role management panel. Currently the AdminDashboard fetches admin users from the `users` table instead.

Column	Type	Notes
id	bigint	PK, auto-generated identity
initials	text	
name	text	NOT NULL
email	text	NOT NULL
role	text	
role_style	text	
last_active	text	
avatar_bg	text	

Pages: AdminDashboard (via `getAdmins` in `dashboardApi.js`, but currently unused — dashboard uses `users` table).

my_courses

Student course enrollments (the student's personal course list).

Column	Type	Notes
id	bigint	PK, auto-generated identity
user_id	bigint	FK → <code>users(id)</code>
code	text	NOT NULL (matches <code>courses.id</code>)
title	text	NOT NULL
description	text	
instructor	text	

progress	integer	
term	text	
status_text	text	
status_type	text	
banner_key	text	Maps to a local SVG banner lookup in the component

Pages: `MyCourses` (read + create via registration), `StudentDashboard` (read for schedule filtering).

`student_metrics`

Dashboard KPI cards for students.

Column	Type	Notes
id	bigint	PK, auto-generated identity
user_id	bigint	FK → users(id)
title	text	NOT NULL
value	text	NOT NULL
trend	text	
progress	integer	
is_alert	boolean	DEFAULT false
icon_key	text	Maps to a lucide-react icon in the component

Pages: `StudentDashboard` (read, filtered by `user_id`).

6. API Layer

All files are in `src/api/`. Each exports plain `async` function calls that use the shared Supabase client from `src/lib/supabaseClient.js`.

`notificationsApi.js`

Function	Table	Description
<code>getNotifications(userId)</code>	<code>notifications</code>	Fetches notifications, optionally filtered by <code>user_id</code>
<code>markNotificationRead(id)</code>	<code>notifications</code>	Sets <code>read = true</code> for a single notification
<code>markAllNotificationsRead(userId)</code>	<code>notifications</code>	Sets <code>read = true</code> for all unread notifications, optionally scoped to a user

`scheduleApi.js`

Function	Table	Description
----------	-------	-------------

getScheduleClasses()	schedule_classes	Fetches all schedule entries
----------------------	------------------	------------------------------

gradesApi.js

Function	Table	Description
getSemesters()	semesters	Fetches all semesters, ordered by id
getSemesterCourses()	semester_courses	Fetches all semester course records, ordered by id

usersApi.js

Function	Table	Description
getUsers()	users	Fetches all users
createUser(user)	users	Inserts a new user, returns the created row
updateUser(id, updates)	users	Updates a user by ID, returns the updated row
deleteUser(id)	users	Deletes a user by ID

coursesApi.js

Function	Table	Description
getCourse(courseId)	courses	Fetches a single course by ID (uses maybeSingle)
getAllCourses()	courses	Fetches all courses
createCourse(course)	courses	Inserts a new course
updateCourse(id, updates)	courses	Updates a course by ID
deleteCourse(id)	courses	Deletes a course by ID
getCourseStudents(courseId)	course_students	Fetches enrolled students for a course
getCourseSessions(courseId)	course_sessions	Fetches upcoming sessions for a course
createCourseSession(session)	course_sessions	Creates a new session
getCourseWeeks(courseId)	course_weeks	Fetches weekly structure for a course
getWeekMaterials(weekId)	course_materials	Fetches materials for a specific week
getCourseFiles(courseId)	course_files	Fetches recent files for a course

gradebookApi.js

Function	Table	Description
getGradebookEntries(courseId)	gradebook_entries	Fetches grade entries for a course
updateGradebookEntry(id, updates)	gradebook_entries	Updates a single grade entry

dashboardApi.js

Function	Table	Description
<code>getInstructorActiveCourses()</code>	<code>instructor_active_courses</code>	Fetches instructor course cards (legacy)
<code>getInstructorAlerts()</code>	<code>instructor_alerts</code>	Fetches instructor sidebar alerts
<code>getStudentMetrics(userId)</code>	<code>student_metrics</code>	Fetches student KPI metrics, optionally filtered by user
<code>getMyCourses(userId)</code>	<code>my_courses</code>	Fetches student's enrolled courses, optionally filtered by user
<code>getAdminStats()</code>	<code>admin_stats</code>	Fetches admin dashboard stat cards (legacy)
<code>getAdmins()</code>	<code>admins</code>	Fetches admin panel users (legacy)

assignmentsApi.js

Function	Table	Description
<code>getAssignments(courseId)</code>	<code>assignments</code>	Fetches assignments for a course, newest first
<code>createAssignment(assignment)</code>	<code>assignments</code>	Creates a new assignment
<code>updateAssignment(id, updates)</code>	<code>assignments</code>	Updates an assignment by ID
<code>deleteAssignment(id)</code>	<code>assignments</code>	Deletes an assignment by ID
<code>getSubmissions(assignmentId, userId)</code>	<code>assignment_submissions</code>	Fetches submissions, optionally filtered by user
<code>submitAssignment(assignmentId, userId, submissionText, fileUrl)</code>	<code>assignment_submissions</code>	Creates a new submission with text and/or file URL
<code>registerForCourse(userId, code, title, instructor, term)</code>	<code>my_courses</code>	Enrolls a student in a course

7. Validation

All schemas are in `src/validation/` and use Zod v4.

loginSchema.js

```
email: string, min length 1 ("Email is required"), must be valid email format
password: string, min length 1 ("Password is required")
```

userSchema.js

```
name: string, min length 1 ("Name is required")
email: string, must be valid email ("Valid email is required")
```

```
role: enum ['student', 'instructor', 'admin']
status: enum ['Active', 'Inactive', 'Offline', 'Pending']
password: string, min length 1 ("Password is required"), optional
phone: string, optional
bio: string, optional
```

courseSchema.js

courseSchema:

```
id: string, min length 1 ("Course ID is required")
title: string, min length 1 ("Title is required")
meta: string, optional
description: string, optional
department: string, optional
term: string, optional
credits: number, optional
instructor_name: string, optional
instructor_role: string, optional
instructor_email: string, must be valid email, optional
instructor_office: string, optional
instructor_hours: string, optional
```

sessionSchema:

```
course_id: string, min length 1 ("Course ID is required")
month: string, min length 1 ("Month is required")
day: string, min length 1 ("Day is required")
title: string, min length 1 ("Title is required")
meta: string, optional
tone: string, optional
```

gradebookSchema.js

```
hw1: number, min 0, max 100, optional, nullable
quiz1: number, min 0, max 50, optional, nullable
midterm: number, min 0, max 200, optional, nullable
hw2: number, min 0, max 100, optional, nullable
attention: boolean, optional
excused: boolean, optional
```

profileSchema.js

```
name: string, min length 1 ("Name is required"), optional
email: string, must be valid email, optional
phone: string, optional
bio: string, optional
```

assignmentSchema.js

assignmentSchema:

```
course_id: string, min length 1 ("Course ID is required")
title: string, min length 1 ("Title is required")
description: string, optional
due_date: string, min length 1 ("Due date is required")
```

submissionSchema:

```
submission_text: string, optional
file_url: string, optional
Refinement: at least one of submission_text or file_url must be provided
("Either submission text or a file is required")
```

8. Pages Reference

LoginPage.jsx

- **Route:** /login
- **Role:** Public
- **Description:** Email/password login form with Zod field validation. On success, redirects to the user's role-specific dashboard.
- **CRUD:** Read-only (reads `users` table via `AuthContext.login`)

StudentDashboard.jsx

- **Route:** /student/dashboard
- **Role:** student
- **Description:** Student home page showing KPI metrics (GPA, credits, missing assignments), today's schedule (filtered to enrolled courses), recent grades, and an action-required box for pending assignments. "Register for Spring" button navigates to course catalog.
- **CRUD:** Read-only

MyCourses.jsx

- **Route:** /student/courses
- **Role:** student
- **Description:** Displays enrolled courses as cards with progress bars and status badges. Includes a "Browse Catalog" toggle that shows available courses from the `courses` table. Students can register for courses, which creates entries in both `my_courses` and `course_students` , and generates a notification.
- **CRUD:** Read + Create (registration)

CourseDetail.jsx

- **Route:** /student/courses/:courseId
- **Role:** student
- **Description:** Detailed course view with tabs: Overview, Materials (weekly accordion with PDF/video/code files), Assignments (list with inline submission form supporting text + file URL), and Grades. Sidebar shows instructor info, course stats, and recent files.
- **CRUD:** Read + Create (assignment submissions)

Notifications.jsx

- **Route:** /student/notifications , /notifications
- **Role:** student (primary), any logged-in user (shared route)

- **Description:** Inbox view grouped by day (Today/Yesterday) with category filters (All, Academic, Campus Events, System). Category counts are computed client-side from fetched data. "Mark all as read" button updates all unread notifications.
- **CRUD:** Read + Update (mark as read)

Schedule.jsx

- **Route:** /student/schedule
- **Role:** student
- **Description:** Weekly timetable with desktop grid view (days × hours) and mobile day-tab list view. Highlights the current day. Class blocks are positioned absolutely based on start/end hours.
- **CRUD:** Read-only

Grades.jsx

- **Route:** /student/grades
- **Role:** student
- **Description:** Transcript page showing cumulative GPA summary card and course history grouped by semester. Shows "N/A" for GPA fields and "No grades recorded yet" when no data exists.
- **CRUD:** Read-only

InstructorDashboard.jsx

- **Route:** /instructor/dashboard
- **Role:** instructor
- **Description:** Instructor home page showing stat cards (total enrolled, pending grading, unread messages), active courses (filtered by `instructor_id`), quick announcement form (sends notifications to enrolled students in a selected course), and a needs-attention sidebar. "Create Assignment" button opens a modal popup with course selection, title, description, and due date fields.
- **CRUD:** Read + Create (announcements, assignments via modal)

CourseManagement.jsx

- **Route:** /instructor/courses/:courseId
- **Role:** instructor
- **Description:** Course management page showing enrolled students table (with search), management action buttons, upcoming sessions, and an assignments section with inline create/delete forms. "New Assignment" button scrolls to the assignments card.
- **CRUD:** Full CRUD (assignments), Read (students, sessions, course info)

Gradebook.jsx

- **Route:** /instructor/courses/:courseId/gradebook
- **Role:** instructor
- **Description:** Grade entry table showing all enrolled students (merged from `gradebook_entries` and `course_students`). Score cells (HW1, Quiz1, Midterm, HW2) are click-to-edit — clicking opens an input that saves via API on blur or Enter. Shows "No students in gradebook yet" when empty.
- **CRUD:** Read + Update (score editing)

AdminDashboard.jsx

- **Route:** /admin/dashboard
- **Role:** admin
- **Description:** System overview with computed stat cards (total students, faculty, active courses from real data), quick action buttons (Add New User → `/admin/users`, Create Course → `/admin/courses/new`,

System Announcement → modal), and a role management table showing admin users. "Manage Roles" button navigates to `/admin/users`.

- **CRUD:** Read + Create (system announcements via modal)

UserManagement.jsx

- **Route:** `/admin/users`
- **Role:** admin
- **Description:** Full user management with search, filter chips, and a create/edit modal. Modal includes name, email, password (required on create, optional on edit), role dropdown, and status dropdown. Desktop table has edit (pencil) and delete (kebab menu dropdown) actions. Mobile layout has card-based view with the same actions. Roles are normalized: stored lowercase in DB, displayed capitalized in UI.
- **CRUD:** Full CRUD

AdminCourseCreate.jsx

- **Route:** `/admin/courses/new`
- **Role:** admin
- **Description:** Course creation form with fields for course ID, title, department, term, credits, and description. Includes an instructor assignment dropdown (fetches users with role='instructor') and a schedule section where admin can add multiple class time entries (day + start/end time). Validates with `courseSchema` and checks for duplicate course IDs before saving. On save, creates the course and inserts schedule entries into `schedule_classes`.
- **CRUD:** Create

Profile.jsx

- **Route:** `/profile`
- **Role:** Any logged-in user
- **Description:** Profile and settings page with tabs: Personal Info (edit form wired to `updateUser`), Security (password change form, 2FA toggle), Notifications (toggle preferences), Appearance (placeholder). All display data (name, email, role) comes from `AuthContext` — no hardcoded values.
- **CRUD:** Edit only (personal info)

9. Known Limitations & Open TODOs

TODO Comments Found in Codebase

File	Line	Comment
src/pages/Profile.jsx	146	// TODO: استبدلها ببناء الـ API 2 السر / لتحديث كلمة السر (Replace with API call for password/2FA update)

Architectural Limitations

1. **Plaintext passwords:** The `users.password` column stores passwords in plaintext. There is no hashing (bcrypt, argon2, etc.). `AuthContext.login()` compares `dbUser.password !== password` directly. This must be replaced with proper password hashing before any production deployment.
2. **No Supabase Auth (GoTrue):** The system uses a custom auth flow querying the `users` table directly instead of Supabase's built-in authentication service. This means no JWT tokens, no refresh tokens, no OAuth providers, and no RLS policies tied to `auth.uid()`.

3. **File attachments are URL-only:** Assignment submissions support a `file_url` text field for linking to external files (Google Drive, Dropbox, etc.). There is no actual file upload to Supabase Storage. A real file upload feature would require setting up a Supabase Storage bucket and implementing upload/download logic.
4. **Legacy tables still exist:** `instructor_active_courses`, `instructor_alerts`, `admin_stats`, and `admins` tables exist in the database and have corresponding API functions in `dashboardApi.js`, but the current dashboard pages compute their data from `users`, `courses`, `course_students`, and `assignments` instead. These legacy tables and API functions are dead code.
5. **No pagination:** `UserManagement` shows a pagination UI but it is non-functional — all users are loaded at once. No API functions accept `limit` / `offset` parameters.
6. **No real-time updates:** The portal does not use Supabase Realtime subscriptions. Data is only fetched on page mount or after explicit user actions.
7. **banner_key hardcoded on registration:** When a student registers for a course via `registerForCourse()`, the `banner_key` is always set to `'computer-science'` regardless of the actual course subject. The `MyCourses` component only has three banner SVGs: `computer-science`, `mathematics`, and `physics`.

10. Setup Instructions

Prerequisites

- Node.js (v18+ recommended)
- A Supabase project with the tables created (run `agent-instructions.md` SQL block, then `migration-v2.sql`)

Environment Variables

Create a `.env` file in the project root (this file is gitignored):

```
VITE_SUPABASE_URL=your_supabase_project_url_here
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key_here
```

The `VITE_` prefix is required — Vite only exposes environment variables to the client bundle if they start with `VITE_`. A `.env.example` file with placeholder values is committed to the repository for reference.

Install & Run

```
npm install
npm run dev
```

The dev server starts at `http://localhost:5173` (Vite default).

Build for Production

```
npm run build
npm run preview
```

Initial Admin Account

After running the SQL migrations, one admin account is seeded:

- **Email:** admin@depi.com
- **Password:** admin123

From the admin dashboard, create instructor and student accounts via User Management. Each new user needs a password set at creation time.